

📖 Model Context Protocol (MCP) — Concise Notes

1. Problem Domain (Why MCP?)

Building Gen AI apps with LLMs + Langchain + custom tools has major issues:

- **Tokens are the currency** we burn — tool definitions alone consume huge numbers of tokens *before* the agent even reads the user's request.
- **Tight Coupling** — everyone has to define tools and functionality every time.
- **Poor interoperability.**
- Every AI App separately wires: AI App → Prompt → Tools → Resources (fragmentation, duplicated effort).

MCP solves this = open standard for connecting AI agents to external systems. Developers implement MCP **once** in their agent → unlocks an entire ecosystem of integrations.

- Launched **November 2024** (by Anthropic).
- Thousands of MCP servers, SDKs for all major languages, now the **de-facto standard**.

2. What is MCP?

- **MCP (Model Context Protocol)** = an **open-source standard** for connecting AI applications to external systems.
- AI apps (Claude, ChatGPT, etc.) can connect to:
 - **Data sources** (local files, databases)
 - **Tools** (search engines, calculators)
 - **Workflows** (specialized prompts)
- **Analogy:** "USB-C port for AI applications" — just as USB-C standardizes connecting devices, MCP standardizes connecting AI apps to external systems.

AI applications side: Chat interface (Claude Desktop, LibreChat), IDEs/code editors (Claude Code, Goose), Other AI apps (Sire, Superinterface). **Data sources & tools side:** Data/file systems (PostgreSQL, SQLite, GDrive), Development tools (Git, Sentry), Productivity tools (Slack, Google Maps). → **Bidirectional data flow** in both directions.

3. Analogous Protocols

- **APIs** → standardize how **web applications** talk to backends/databases. Common language for data exchange.
- **LSP (Language Server Protocol)** → standardizes how **IDEs** interact with programming language tools. Write one language server → works in VS Code, Vim, or any LSP-compatible editor.
- **MCP** → does the same for AI apps, standardizing **three critical elements**:
 - **Prompts** — Reusable templates for common tasks
 - **Tools** — Functions that let the AI take actions
 - **Data & Resources** — Ways to access and understand information

(The slide card also lists a 4th: **Sampling**.)

4. Key Benefits of MCP

- **Customize any AI app** (Claude or your own) — from local files to cloud apps to internal knowledge bases.
- **Write once, work everywhere** — you write one MCP server, any AI application can use it (just like one REST API → any web client).
- **Clearly separated development** — like front-end vs back-end. Teams can specialize as either MCP client builders or MCP server builders.

5. How It Works (Client ↔ Server)

- **MCP Client** — Queries for Resources, Invokes Tools, Interpolates Prompts.
- **MCP Server** — Exposes Resources, Exposes Tools, Exposes Prompt Templates.

What a server can expose:

Primitive	Description
Tools	Most common — CRUD on data, Git ops, CRM actions
Resources	Raw data endpoints / RAG queries the model can pull
Prompt templates	Reusable system or task prompts the client can insert
Sampling calls	Server can ask the client/LLM to do work on its behalf (rare but powerful)

Current State: Lots of tool-only servers; very few use Resources & Prompt Templates → **ripe area to explore/contribute**. **Key Takeaway:** Don't just build another tool server — try exposing curated RAG queries or prompt packs (low competition, high impact).

6. MCP: Standardized AI Development (Flow)

- **Client column (LLM lives here)** — any of these (Claude for Desktop, VS Code, AI Assistants/Copilots) can host the model & decide when to fire a tool call.
- **Server column (tools exposed)** — e.g., Data Store MCP server, CRM MCP server, Version Control MCP server (e.g., Git).
- **Back-end targets** — e.g., Data Stores (DynamoDB, S3, Aurora), CRM systems, Version Control Software.

Flow: Client ⇔ picks tool ⇔ sends request to server ⇔ server bridges to real system ⇔ result back to model.

Key takeaway: Swap clients or servers independently — **MCP is the USB-C that keeps them interoperable**.

7. MCP Architecture — Key Participants

- **MCP Host** — The AI application that **coordinates and manages** one or multiple MCP clients.

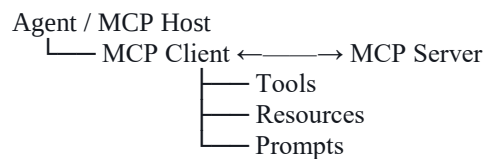
- **MCP Client** — A component that **maintains a connection to an MCP server** and obtains context from it for the MCP host to use.
- **MCP Server** — A program that **provides context** to MCP clients.

Example: VS Code = MCP host. When it connects to **Sentry MCP server**, the runtime instantiates an MCP client object. When it also connects to the **local filesystem server**, another MCP client object is created. → **One MCP client per MCP server**.

Note: *MCP server* refers to the application/program that serves context data, regardless of where it runs:

- **Local MCP server** — runs on same machine (e.g., filesystem server launched by Claude Desktop → uses **STDIO transport**).
- **Remote MCP server** — runs on a remote platform (e.g., Sentry's official server → uses **Streamable HTTP transport**).

8. Architecture Diagram Summary



9. Transport Layer

- A "**transport**" = the pipe that carries messages back and forth.
- MCP is based on **JSON-RPC 2.0** — messages are structured text; transport decides *how* that text travels.
- The transport layer manages **communication channels, authentication, connection establishment, message framing, and secure communication** between participants.
- **Two primary transports: stdio and SSE (Server-Sent Events).**
- Local MCP servers using **STDIO** typically serve a **single MCP client**; remote MCP servers using **Streamable HTTP** typically serve **many MCP clients**.

a) stdio (Standard Input/Output) — LOCAL transport

- When an MCP client wants to use a tool, it **starts the MCP server as a child process**.
- **The Pipe:** Sends JSON to server's **stdin**, reads responses from server's **stdout**.
- **Why use it?**
 - **Incredibly fast** — no network overhead.
 - **Very secure** — server only exists while client is running it.
- Uses standard input/output streams for direct process communication between local processes on the same machine.

b) SSE (Server-Sent Events) — REMOTE transport

- Built on **HTTP**.
- **The Pipe:** Client opens a **long-running connection** to an HTTP endpoint to receive a **stream of messages** from the server. To send messages to the server → standard **HTTP POST requests**.
- **Why use it?** Talks to tools running in the cloud / on a different server.
- Supports standard HTTP auth methods: **bearer tokens, API keys, custom headers**.
- **MCP recommends using OAuth** to obtain authentication tokens.

10. Langchain MCP Library — Example

python

```
from langchain_mcp_adapters.client import MultiServerMCPClient
```

```
client = MultiServerMCPClient({
    "local_server": {
        "transport": "stdio",
        "command": "python",
        "args": ["<mcp_server_file_path>.py"],
    }
})
```

11. Data Layer Protocol — JSON-RPC 2.0

- MCP uses **JSON-RPC 2.0** as its foundational language — a lightweight protocol for one system to "call" a function on another and get a structured result back.
- **Stateless, light-weight remote procedure call protocol.** Transport agnostic (works within the same process, over sockets, HTTP, or various message passing environments).
- An RPC call = a **Request object** sent to a Server.

Every action follows a strict structure — Request object members:

Field	Description
jsonrpc	String specifying the version. MUST be exactly "2.0" .
method	String containing the name of the method to invoke (e.g., <code>tools/call</code>). Names starting with <code>rpc.</code> are reserved.
params	Structured value holding parameter values for the invocation.
id	Identifier established by the Client — MUST contain String, Number, or NULL. If not included → assumed to be a notification.

Other features:

- **ID** — matches a specific response to its original request.
- **Notifications** — server or client can send "one-way" updates (like a progress report) **without requiring a formal response**. Example: when a server's available tools change, the server sends tool update notifications. Sent as JSON-RPC 2.0 notification messages.

- **Standardized error codes** — so the AI knows if a failure was due to a connection issue, missing tool, or invalid inputs.

12. MCP Server Primitives (The 3 Main Types)

The data layer organizes information into **three main "Primitives"** — three types of things an AI can "see" or "do" through the protocol:

Primitive	Control	Description	Example
Prompts 	User-controlled	Interactive templates invoked by user choice	Slash commands, menu options
Resources 	Application-controlled	Contextual data attached and managed by the client	File contents, git history
Tools 	Model-controlled	Functions exposed to the LLM to take actions	API POST requests, file writing

- **Prompts** — Pre-defined templates that help the LLM understand how to interact with a specific set of data.
- **Resources** — Read-only data (file contents, database row). Structured data or content that provides additional context to the model.
- **Tools** — Executable functions that allow models to perform actions or retrieve information.

13. Primitive Methods (Discovery / Retrieval / Execution)

Each primitive type has associated methods:

- `*/list` → **discovery**
- `*/get` → **retrieval**
- `tools/call` → **execution** (for tools)

Clients use `*/list` methods to discover available primitives — e.g., a client first lists all available tools (`tools/list`), then executes them. **Listings are dynamic.**

Example: An MCP server providing context about a database can expose:

- **Tools** for querying the database
- A **Resource** containing the schema of the database
- A **Prompt** that includes few-shot examples for interacting with the tools

14. Slash Commands

- In MCP, **slash commands** = a **UI-level, structured way** for a client (chat app, IDE, Copilot UI) to expose predefined actions backed by MCP server capabilities.
- **Not part of the MCP wire protocol itself** — they are a **convention implemented by MCP clients** on top of MCP tools, prompts, or resources.

- A shortcut command (starting with /) the user types → client translates into a specific MCP action (invoke a tool, load a prompt, or fetch a resource).

User view: /summarize, /debug, /explain **MCP view:** → invoke a tool / load a predefined prompt / call a server capability.

15. Server Primitives in Detail

🔗 Prompts (slash commands)

- Designed to be **user-controlled** — exposed from servers to clients with the intention of the user being able to **explicitly select them** for use.
- Example in VSCode: prompts from Copilot, mslearn, Git, Azure MCP servers shown in the slash menu (e.g., /create-instructions, /create-skill, /create-agent, /azure-ai, /azure-compliance).

🔗 Resources

- **Read-only, URI-addressable context** (files, schema, configs, logs, snapshots) that clients can **fetch and attach** to model context.
- Spec defines `resources/list` (discovery) and `resources/read` (retrieval).

Example: "Data Platform Helper" MCP server exposes 3 resources:

- `data://config` → returns JSON config (read-only, safe subset)
- `schema://{table}` → returns a dynamic schema string for a table
- `runlog://latest` → returns latest pipeline run log excerpt

Used for LLM questions like: "What columns does silver_orders have?", "What is the current ingestion mode?", "Why did the last run fail?"

16. MCP SDKs (As of March 2026)

Tier 1	Tier 2	Tier 3	TBD
TypeScript, Python, C#, Go	Java, Rust	Swift, Ruby, PHP	Kotlin

Quiz Questions

◆ B. Technical / Code-Based

Q33. Fill in the missing transport value:

```
python
client = MultiServerMCPClient({
    "local_server": {
```

```

    "transport": "____",
    "command": "python",
    "args": ["<mcp_server_file_path>.py"],
  }
})

```

Answer: "stdio".

Q34. What class from `langchain_mcp_adapters.client` is used to connect to MCP servers?

Answer: `MultiServerMCPClient`.

Q35. Given the Data Platform Helper MCP server exposes resources, what URI retrieves the schema of a table called `silver_orders`? **Answer:** `schema://silver_orders`.

Q36. Write a valid JSON-RPC 2.0 Request for calling a tool named `add` with arguments `a=12`, `b=7`, request id 1. **Answer:**

```

json
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": { "name": "add", "arguments": { "a": 12, "b": 7 } }
}

```

Q37. What method does a client call first to discover all available tools on a server? **Answer:** `tools/list`.

Q38. Suppose an MCP server provides DB context. What 3 primitive types could it expose, and for what purpose? **Answer:**

- **Tools** → querying the database.
- **Resource** → containing the schema of the database.
- **Prompt** → including few-shot examples for interacting with the tools.

◆ C. Output-Based / Scenario

Q39. A JSON-RPC message is sent without an `id` field. What does the server interpret this as? **Answer:** A **notification** — one-way message, no response expected.

Q40. Claude Desktop launches a local filesystem server. Which transport is used, and why is it fast and secure? **Answer:** **STDIO transport** — fast because there's **no network overhead**, and secure because the **server only exists as long as the client runs it** (child process).

Q41. The Sentry MCP server runs on Sentry's platform and is accessed over HTTP. This is a _____ MCP server using the _____ transport. **Answer:** **remote**; **Streamable HTTP**.

Q42. A user types `/summarize` in a chat app. From MCP's point of view, what happens?

Answer: The client translates the slash command into an MCP action — e.g., invoking a tool, loading a predefined prompt, or calling a server capability.

Q43. The `jsonrpc` field is set to `"1.0"` in a request. Is this valid? **Answer: No** — `jsonrpc` MUST be **exactly** `"2.0"`.

Q44. You are building an MCP server and want to maximize impact given the state of the ecosystem. Which primitives should you focus on and why? **Answer:** Focus on **Resources and Prompt Templates** — the ecosystem has lots of tool-only servers, but very few use Resources & Prompt Templates → **low competition, high impact**. Specifically, curated RAG queries or prompt packs.

Q45. Why is "MCP is the USB-C that keeps them interoperable" important? **Answer:** Because you can **swap clients or servers independently** — any MCP client can talk to any MCP server using the same standardized protocol.

◆ D. Quiz-POV Rapid Fire

- **MCP launched when & by whom?** → November 2024, by Anthropic.
- **MCP's underlying RPC protocol?** → JSON-RPC 2.0.
- **Two primary MCP transports?** → stdio and SSE.
- **Three MCP primitives?** → Prompts, Resources, Tools.
- **MCP Host example?** → VS Code / Claude Desktop / Claude Code.
- **How many MCP clients per MCP server connection?** → One.
- **Which primitive is user-controlled?** → Prompts.
- **Which primitive is model-controlled?** → Tools.
- **Which primitive is application-controlled?** → Resources.
- **What does `*/list` do?** → Discovery of primitives.
- **What does `tools/call` do?** → Executes a tool.
- **Are slash commands part of the MCP wire protocol?** → No — they are a client-side UI convention.
- **Recommended auth for HTTP transport?** → OAuth.
- **Resources are read-only?** → Yes — URI-addressable, read-only context.
- **Tier 1 SDKs?** → TypeScript, Python, C#, Go.
- **True/False: MCP notifications expect a response.** → False — they are one-way.
- **"USB-C for AI applications" means?** → standardized way to connect AI apps to external systems.